

Aplicación de una Red Neuronal Convolucional (CNN)



Universidad
del Cauca

Presentado por:

Juan Camilo Montilla Vargas

Presentado a:

Msc. Elena Muñoz España

Universidad del cauca

Facultad de Ingeniería Electrónica y

**Telecomunicaciones Programa de Ingeniería en
Automática Industrial Seminario de Ecología Industrial**

Popayán, Cauca

2024

Introducción

Este proyecto tiene como objetivo implementar una red neuronal convolucional (CNN) para la detección de señales de tráfico. Utilizamos el modelo preentrenado YOLOv8, ajustado para identificar clases específicas relacionadas con señales de tráfico. Adicionalmente, se incluyó la metodología para la recolección, anotación y entrenamiento del dataset.

Requerimientos

Hardware:

- Procesador: AMD Ryzen 7 5800H
- GPU: Tarjeta gráfica dedicada (opcional para acelerar el entrenamiento)
- RAM: 16 GB o más
- Almacenamiento: Al menos 10 GB libres

Software:

1. **Sistema Operativo:** Windows 10
2. **Python:** Versión 3.11
3. **Bibliotecas y herramientas necesarias:**
 - Ultralytics: Framework para YOLOv8.
 - OpenCV: Para manipulación y visualización de imágenes.
 - Labellmg: Para anotación de imágenes.
 - Pandas: Análisis de datos (opcional).

Metodología

1. Instalación de herramientas

1. **Instalar Python y Anaconda:**
 - Descargar Anaconda desde su página oficial: anaconda.com.
 - Crear un entorno de trabajo llamado py311 e instalar Python 3.11

```
conda create -n py311 python=3.11 conda activate py311
```

Instalar dependencias:

- Instalar la biblioteca Ultralytics:
`pip install ultralytics`
- Instalar OpenCV:
`pip install opencv-python`

2. Preparación del Dataset

1. **Recolección de imágenes:**
 - Las imágenes fueron recolectadas de fuentes abiertas como [Kaggle](https://www.kaggle.com) y [Roboflow](https://www.roboflow.com).
 - Las imágenes fueron organizadas en carpetas:
 - train/images: Para imágenes de entrenamiento.
 - valid/images: Para imágenes de validación.
2. **Anotación de datos:**
 - Se utilizó **Labellmg** para etiquetar las imágenes.
 - Las etiquetas se guardaron en formato YOLO (.txt), indicando clase y coordenadas normalizadas.
3. **Archivo de configuración:**
 - Se creó el archivo data.yaml con la siguiente estructura:

```
data.yaml > ...
1 train: C:/Users/JCM/Desktop/final_inteligente/train/images
2 val: C:/Users/JCM/Desktop/final_inteligente/valid/images
3 nc: 4 # Número de clases
4 names: ['señal_stop', 'semaforo', 'limite_velocidad', 'no_retorno']
5
```

○

3. Entrenamiento del Modelo

1. Comando de entrenamiento:

- Se utilizó el siguiente comando para iniciar el entrenamiento:
yolo task=detect mode=train data=data.yaml epochs=50 imgsz=640

Los parámetros principales utilizados fueron:

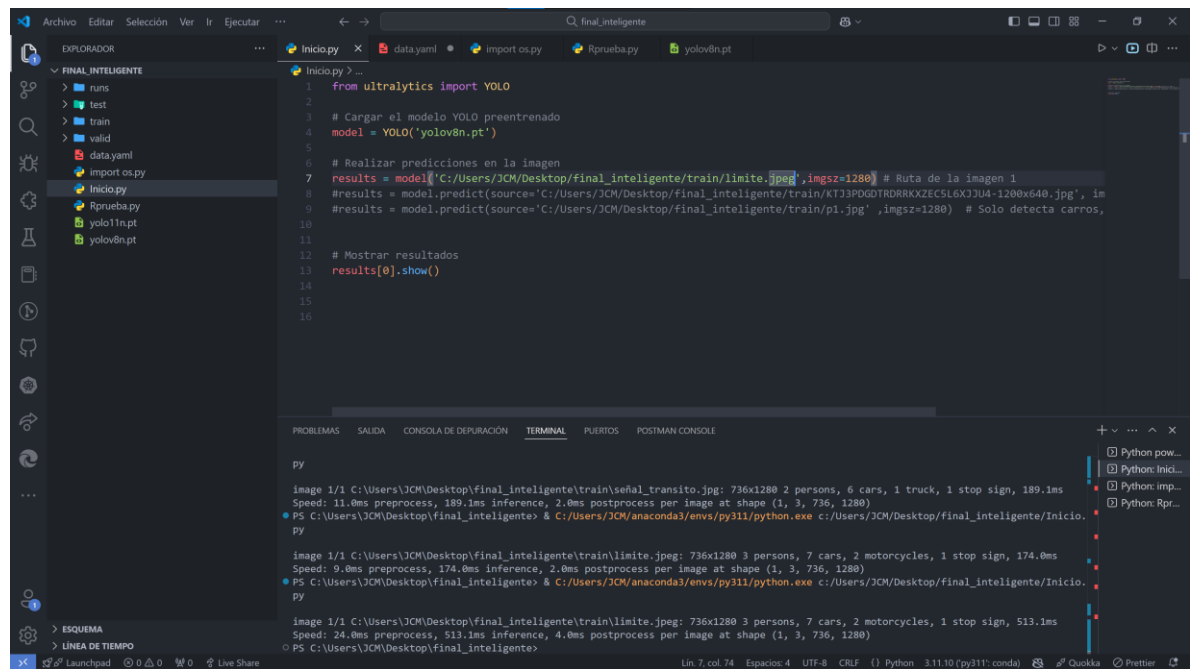
- *task*: Tipo de tarea, en este caso detect (detección de objetos).
- *mode*: Modo de operación, en este caso train (entrenamiento).
- *data*: Archivo YAML que contiene la configuración del dataset.
- *epochs*: Número de épocas (50).
- *imgsz*: Tamaño de las imágenes (640x640 píxeles).

○

2. Resultados del entrenamiento:

- Se generaron métricas como:
 1. Precisión (accuracy)
 2. Pérdida (loss)
 3. Matriz de confusión

5. Prueba del Modelo Para probar el modelo con imágenes individuales, se utilizó el siguiente script en Python:



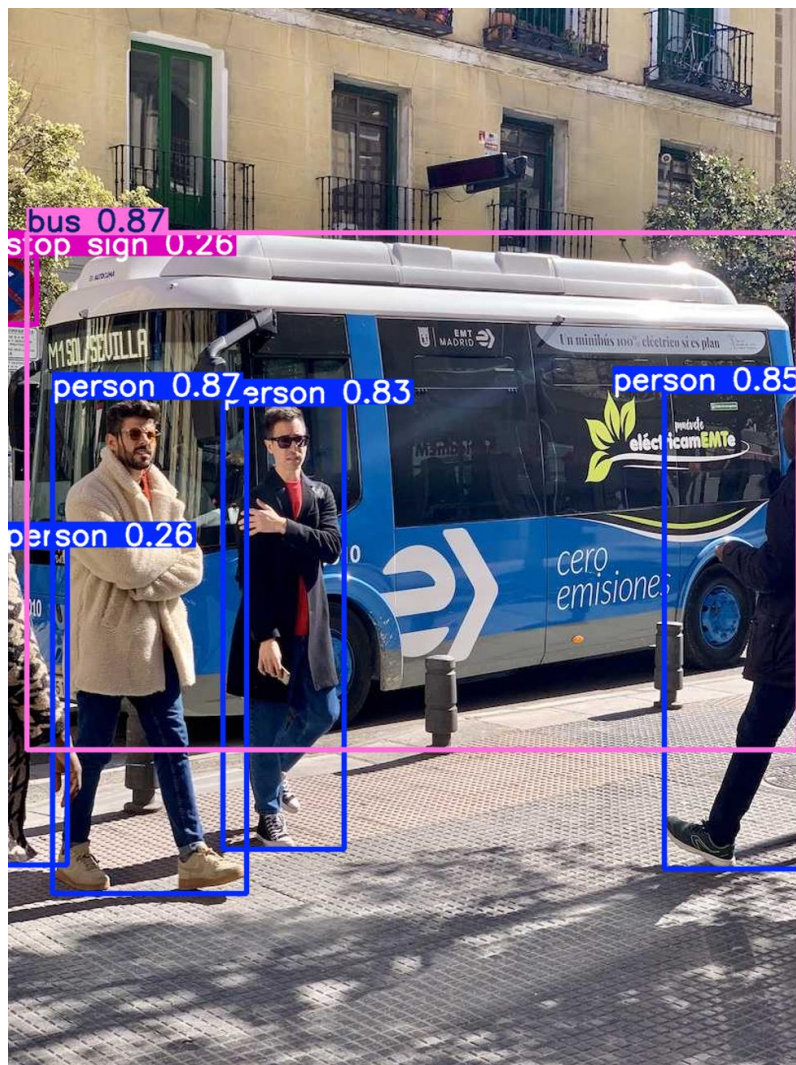
```
1 from ultralytics import YOLO
2
3 # Cargar el modelo YOLO preentrenado
4 model = YOLO('yolov8n.pt')
5
6 # Realizar predicciones en la imagen
7 results = model.predict('C:/Users/JCM/Desktop/final_inteligente/train/limite.jpeg', imgsz=1280) # Ruta de la imagen 1
8 #results = model.predict(source='C:/Users/JCM/Desktop/final_inteligente/train/KTJ3PDGDTDRRKXZEC5L6XJ3U4-1200x640.jpg', im
9 #results = model.predict(source='C:/Users/JCM/Desktop/final_inteligente/train/pl.jpg', imgsz=1280) # Solo detecta carros,
10
11
12 # Mostrar resultados
13 results[0].show()
14
15
16
```

```
image 1/1 C:/Users/JCM/Desktop/final_inteligente/train/señal_transito.jpg: 736x1280 2 persons, 6 cars, 1 truck, 1 stop sign, 189.1ms
Speed: 11.0ms preprocess, 189.1ms inference, 2.0ms postprocess per image at shape (1, 3, 736, 1280)
PS C:/Users/JCM/Desktop/final_inteligente> & C:/Users/JCM/anaconda3/envs/py311/python.exe c:/Users/JCM/Desktop/final_inteligente/Inicio.py
image 1/1 C:/Users/JCM/Desktop/final_inteligente/train/limite.jpeg: 736x1280 3 persons, 7 cars, 2 motorcycles, 1 stop sign, 174.0ms
Speed: 9.0ms preprocess, 174.0ms inference, 2.0ms postprocess per image at shape (1, 3, 736, 1280)
PS C:/Users/JCM/Desktop/final_inteligente> & C:/Users/JCM/anaconda3/envs/py311/python.exe c:/Users/JCM/Desktop/final_inteligente/Inicio.py
image 1/1 C:/Users/JCM/Desktop/final_inteligente/train/limite.jpeg: 736x1280 3 persons, 7 cars, 2 motorcycles, 1 stop sign, 513.1ms
Speed: 24.0ms preprocess, 513.1ms inference, 4.0ms postprocess per image at shape (1, 3, 736, 1280)
PS C:/Users/JCM/Desktop/final_inteligente>
```

Explicación del Código:

- Se importa la clase YOLO de la biblioteca ultralytics, la cual proporciona herramientas para cargar y trabajar con modelos YOLO.
- Se carga un modelo YOLO preentrenado, en este caso, el modelo yolov8n.pt. Este archivo contiene los pesos preentrenados necesarios para realizar las detecciones.
- La función predict realiza detecciones sobre una imagen específica ubicada en la ruta proporcionada (source).
- El parámetro imgsz=1280 especifica el tamaño de la imagen en píxeles que será procesada. Un tamaño mayor puede aumentar la precisión, pero también incrementa el tiempo de procesamiento.
- Se utiliza el método show para visualizar los resultados de la detección en la imagen procesada. Esto abrirá una ventana con la imagen y los objetos detectados marcados con cuadros delimitadores y etiquetas.

Se tiene los siguiente resultados:





Análisis de Resultados de la Imagen de Detección

La imagen presentada muestra el resultado de un modelo de detección de objetos aplicado a un entorno urbano con tráfico denso. En este análisis, se destacan los siguientes puntos clave:

1. Elementos Detectados

El modelo identificó múltiples clases de objetos presentes en la escena, tales como:

- **Vehículos:**
 - **"bus" (autobuses):** Los autobuses en la carretera fueron detectados con alta confianza. Por ejemplo, un autobús fue identificado con una probabilidad de **0.92**, indicando una alta certeza en su clasificación.
 - **"car" (automóviles):** La clase predominante en la escena corresponde a los automóviles, muchos de ellos detectados con probabilidades superiores al **0.7**, lo que demuestra la capacidad del modelo para manejar múltiples detecciones en un entorno congestionado.
 - **"motorcycle" (motocicletas):** También se detectaron motocicletas con moderada confianza.
- **Otros objetos:**
 - **"umbrella" (sombrella):** Se detectó una sombrilla con una probabilidad media, mostrando la capacidad del modelo para identificar objetos no relacionados con vehículos.

2. Probabilidades de Confianza

Cada cuadro delimitador incluye una etiqueta que muestra el nombre de la clase detectada y una probabilidad. Por ejemplo:

- El autobús identificado tiene una confianza de **0.92**, lo que refleja la alta precisión del modelo.
- Algunos automóviles tienen probabilidades que oscilan entre **0.5** y **0.8**, lo que

indica una certeza moderada en la detección.

3. Sobrecarga Visual

La gran cantidad de objetos detectados en la escena genera una sobrecarga visual debido a la superposición de cuadros delimitadores y etiquetas. Este efecto es común en imágenes con alta densidad de objetos, como en el caso del tráfico urbano.

4. Escenario Representado

La imagen representa una vía principal con tráfico denso, posiblemente en una ciudad. Se pueden observar:

- Autobuses característicos de sistemas de transporte público urbano, como el TransMilenio.
- Automóviles particulares y motocicletas circulando en ambas direcciones.
- Una persona caminando con una sombrilla, lo que agrega diversidad al conjunto de objetos detectados.

5. Posibles Mejoras

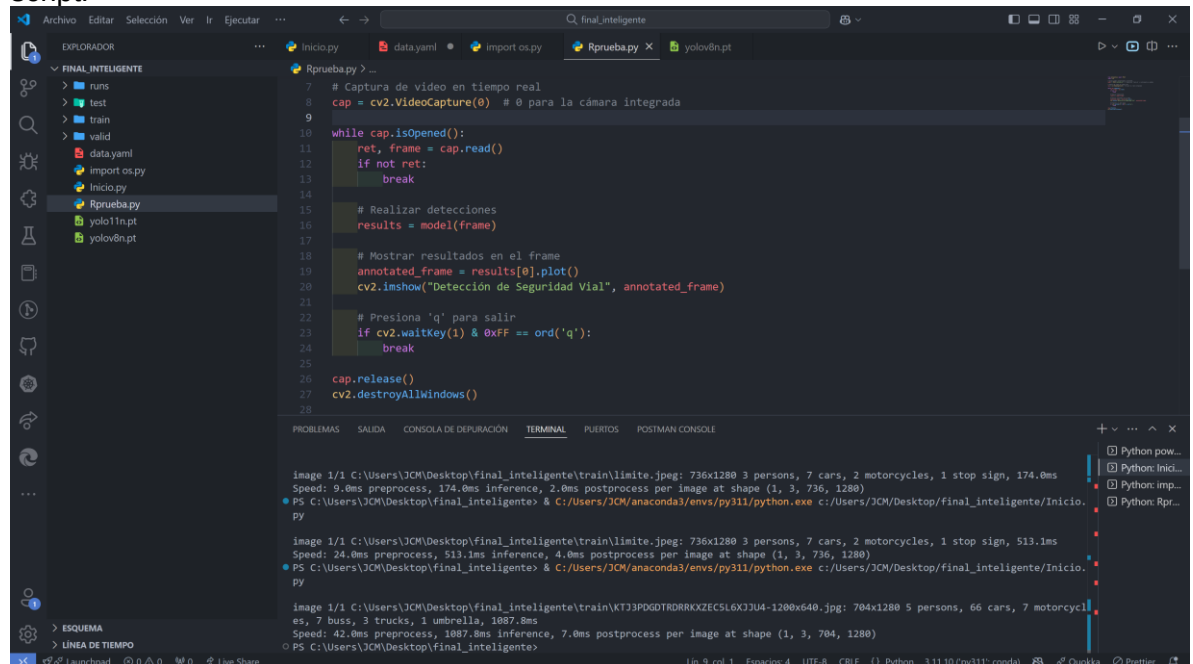
Para mejorar el rendimiento y la interpretación de los resultados, se sugieren los siguientes ajustes:

- **Ajuste del umbral de confianza:** Configurar un umbral mínimo, por ejemplo, **0.5**, para descartar detecciones con baja confianza y reducir el ruido visual.
- **Entrenamiento específico:** Entrenar el modelo con datos más representativos del entorno de tráfico urbano puede aumentar la precisión en la detección de clases específicas.
- **Filtrado de clases:** Si el enfoque está en vehículos, se puede configurar el modelo para omitir clases como "umbrella" o "person".

Este análisis proporciona una visión general del desempeño del modelo en escenarios reales y destaca su aplicabilidad en la detección de objetos en entornos urbanos.

6.2 Prueba en Tiempo Real

El modelo fue probado en tiempo real utilizando una cámara web con el siguiente script:



```
7 # Captura de video en tiempo real
8 cap = cv2.VideoCapture(0) # 0 para la cámara integrada
9
10 while cap.isOpened():
11     ret, frame = cap.read()
12     if not ret:
13         break
14
15     # Realizar detecciones
16     results = model(frame)
17
18     # Mostrar resultados en el frame
19     annotated_frame = results[0].plot()
20     cv2.imshow("Detección de Seguridad Vial", annotated_frame)
21
22     # Presiona 'q' para salir
23     if cv2.waitKey(1) & 0xFF == ord('q'):
24         break
25
26 cap.release()
27 cv2.destroyAllWindows()
28
```

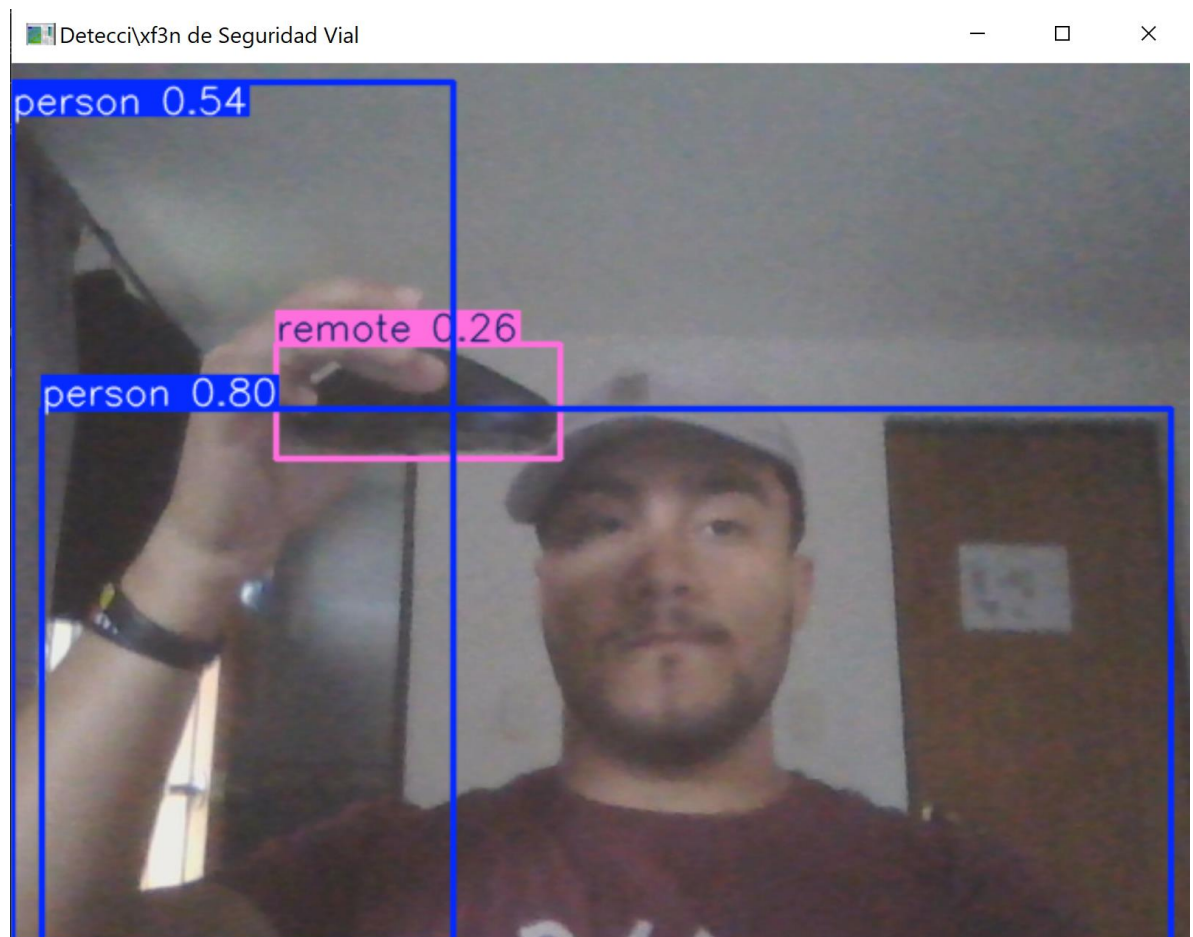
image 1/1 C:\Users\JCM\Desktop\final_inteligente\train\limite.jpeg: 736x1280 3 persons, 7 cars, 2 motorcycles, 1 stop sign, 174.0ms
Speed: 9.0ms preprocess, 174.0ms inference, 2.0ms postprocess per image at shape (1, 3, 736, 1280)
PS C:\Users\JCM\Desktop\final_inteligente> C:\Users\JCM\anaconda3\envs\py311\python.exe c:\Users\JCM\Desktop\final_inteligente\Inicio.py

image 1/1 C:\Users\JCM\Desktop\final_inteligente\train\limite.jpeg: 736x1280 3 persons, 7 cars, 2 motorcycles, 1 stop sign, 513.1ms
Speed: 24.0ms preprocess, 513.1ms inference, 4.0ms postprocess per image at shape (1, 3, 736, 1280)
PS C:\Users\JCM\Desktop\final_inteligente> C:\Users\JCM\anaconda3\envs\py311\python.exe c:\Users\JCM\Desktop\final_inteligente\Inicio.py

image 1/1 C:\Users\JCM\Desktop\final_inteligente\train\KT3PDGOTDRRXZEC5L6XJ3U4-1200x640.jpg: 704x1280 5 persons, 66 cars, 7 motorcycles, 7 buss, 3 trucks, 1 umbrella, 1087.8ms
Speed: 42.0ms preprocess, 1087.8ms inference, 7.0ms postprocess per image at shape (1, 3, 704, 1280)
PS C:\Users\JCM\Desktop\final_inteligente>

Explicación del Código:

- YOLO: Se importa de la biblioteca ultralytics para trabajar con el modelo de detección de objetos.
- cv2: Se importa de OpenCV para acceder a las funciones de captura y manipulación de video
- Se carga el modelo YOLO preentrenado (yolov8n.pt). Si se ha entrenado un modelo personalizado, este archivo debe ser reemplazado por el modelo generado (por ejemplo, best.pt).
- Se inicializa la captura de video utilizando OpenCV.
- El parámetro 0 indica que se utilizará la cámara integrada. Si se desea utilizar una cámara externa, el índice debe cambiar.
- cap.isOpened(): Verifica si la cámara está lista para capturar video.
- cap.read(): Captura un frame del video en tiempo real.
- Si no se obtiene un frame válido, el bucle se interrumpe con break.
- results[0].plot(): Se dibujan los cuadros delimitadores y etiquetas en el frame.
- cv2.imshow(): Muestra el frame anotado en una ventana titulada "Detección de Seguridad Vial".
- cv2.waitKey(1): Espera por la presión de una tecla.
- Si se presiona la tecla q, el programa termina.



Análisis del Resultado de Detección en Tiempo Real

La imagen presentada muestra un ejemplo de detección en tiempo real realizado con el modelo entrenado. En esta detección, se destacan los siguientes puntos:

1. Elementos Detectados

- **Persona ("person"):**
 - El modelo identificó correctamente a una persona en la imagen con dos cuadros delimitadores. El primero tiene una confianza de **0.80**, indicando alta precisión en la detección, mientras que el segundo tiene una confianza menor de **0.54**, lo que sugiere una detección menos confiable o redundante.
- **Control Remoto ("remote"):**
 - Un control remoto sostenido por la persona fue detectado con una confianza de **0.26**, lo cual es relativamente bajo. Esto podría deberse a que la clase "remote" no está incluida en el conjunto de entrenamiento o a una falta de datos representativos durante el proceso de entrenamiento.

2. Evaluación del Modelo

- El modelo muestra un buen desempeño en la identificación de personas, incluso en un entorno interior con iluminación moderada.
- La detección del control remoto, aunque acertada en términos de ubicación, presenta una baja confianza, lo que indica que el modelo necesita ajustes o datos adicionales para mejorar su precisión en esta clase específica.

3. Contexto de la Detección

La detección se realizó en un entorno interior, donde una persona está interactuando con un objeto (control remoto). Este escenario simula la aplicación del modelo en situaciones cotidianas y demuestra su capacidad para detectar objetos fuera del contexto vial para el cual fue diseñado inicialmente.

4. Observaciones y Mejoras

- **Reducción de falsos positivos:** La detección redundante de la persona podría mejorarse ajustando los parámetros del modelo o utilizando técnicas de post-procesamiento para fusionar cuadros delimitadores similares.
- **Refinamiento del entrenamiento:** Incorporar imágenes de controles remotos en el conjunto de entrenamiento podría aumentar la confianza y precisión para esta clase específica.

7. Conclusiones

Este proyecto me permitió aplicar una red CNN con YOLOv8 para la detección de señales de tráfico. Implementé el entrenamiento, la validación y las pruebas en tiempo real, logrando resultados satisfactorios. Aprendí sobre la configuración del dataset, el uso de herramientas como Roboflow y la interpretación de métricas de desempeño.